

Project Part B: Playing the Game

We’ve structured our report into a two-act play. We begin with the details of our final agent in [Section 1](#). Then, in [Section 2](#), we describe the journey there.

1 The Destination

Our submitted agent is `MmCppNnue2`, a deterministic adversarial search agent with a quantised efficiently updatable neural network ($\mathcal{E}\mathcal{U}\mathcal{I}\mathcal{N}$) leaf evaluator. On each turn, it searches forward from the current state under the usual zero-sum assumption that both players choose their best replies, then returns the best action from the deepest completed iteration. The native C++ searcher owns move generation, search, repetition handling, time management, transposition-table storage, and $\mathcal{E}\mathcal{U}\mathcal{I}\mathcal{N}$ inference; the Python wrapper loads weights, passes through the referee clock, and converts between referee actions and compact native moves.

1.1 Search algorithm

The core algorithm is a depth-limited [negamax](#) search, a compact form of [minimax](#), with [alpha-beta pruning](#). The implementation permits depths up to 64, but the clock normally stops the search much earlier. Negamax is a natural fit because positions are always scored from the side to move: after making a move, the child score is negated. Alpha-beta pruning is essential in Cascade because the branching factor is large, especially during placement and in positions with many movement and capture options.

At the root, the search uses [iterative deepening](#): it first searches depth 1, then depth 2, and so on until the time manager decides that another iteration is unlikely to finish. This keeps a fully searched move available if time expires, and earlier iterations provide a strong principal-variation move for later move ordering. The root also uses [principal variation search](#): the first move is searched with a full alpha-beta window, while later moves are tested with a narrow zero window and re-searched fully only when they appear capable of improving alpha. From depth 3 onward, the root initially searches with an aspiration window of 50 centipawn-like units around the previous iteration’s score, widening the window until either the score fits or the window reaches 800. This is effective when move ordering is good, because most non-principal moves fail low cheaply.

Terminal wins use a base score of 1000000 with a ply adjustment. A quicker win is better than a slower win, and a slower loss is better than a faster loss. Draws from threefold repetition are given a contempt score of -10 from the root player’s perspective. This is a practical bias: against weaker agents, accepting a draw too eagerly can throw away positions where a win is still achievable. The contempt value is small enough that the search will still choose a draw over a genuinely losing line. At the root, moves that immediately revisit a previous real-game position receive a small 5-point shuttle penalty, breaking ties against pointless repetition.

1.2 Advanced search heuristics

The search includes several chess-engine-style enhancements, adapted to Cascade. Good references for these techniques are the Chessprogramming Wiki pages on [transposition tables](#), [quiescence search](#), [null-move pruning](#), [late move reductions](#), [reverse futility pruning](#), [ProbCut](#), and [singular extensions](#). The overall philosophy is similar to modern engines such as [Stockfish](#): spend full effort on promising, forcing, or unstable lines, and be selective with quiet moves that are unlikely to change the result.

The most important heuristic is move ordering. The search tries the transposition-table move first, then orders tactical actions by approximate capture value and quiet moves by killer moves,

counter-moves, and history scores. Killer moves are quiet moves that previously caused a beta cutoff at the same remaining depth. The history and tactical-history tables are capped at 3000 and aged by halving when saturated; the counter-move bonus is 4250, placing it below the first killer move but above ordinary history moves. We also apply a bonus to “opposition”: a quiet MOVE that lands diagonally adjacent to an enemy stack (of lesser than or equal value) or one that escapes such a position (when flipped). Such moves receive a 5100 bonus, placing them above the first killer move, and are excluded from late move and futility pruning so the search has to evaluate them. These heuristics do not change the minimax value directly, they make alpha-beta pruning more effective by finding good moves earlier.

The search also uses a large clustered transposition table keyed by [Zobrist hashes](#). Repeated positions can occur naturally in Cascade, so caching is valuable. The table has 2^{20} clusters with four slots per cluster, for 4194304 entries and about 192MiB of storage. Table entries store the searched depth, score bound type, best move, and a cached static evaluation when it fits safely in 16 bits; replacement prefers empty or shallower slots. The cached evaluation avoids repeating EUIII inference when a position is revisited for pruning tests such as reverse futility pruning or null-move pruning.

At depth zero, the agent does not immediately call the static evaluator. Instead, it enters quiescence search over tactical moves, mainly captures and profitable cascades. This reduces the horizon effect: a quiet-looking leaf where the opponent has an obvious capture available should not be evaluated as stable. The quiescence search is bounded to eight plies and uses a delta-pruning margin of 200, so implausible tactical moves are skipped when even their best likely gain cannot raise alpha.

Several forward-pruning and reduction techniques control the search width. Reverse futility pruning applies up to depth 5 with a margin of 110 per remaining ply. Null-move pruning applies in play-phase non-principal nodes from depth 3 onward, with a base reduction of $3 + \text{depth}/5$ and extra reduction when the static score is far above beta. Razoring applies only at depth 1, using a 200-point margin before dropping to quiescence search. ProbCut applies from depth 6, searching forcing tactical moves at depth reduced by 4 against a beta margin of 180. Late-move pruning skips the quiet tail in shallow non-principal nodes using thresholds $\{3, 5, 8, 13, 20, 30, 42\}$ for depths 1 through 7, keeping extra width near the root where pruning mistakes are most costly. Late-move reductions search late or low-priority moves at reduced depth first and only re-search them if they improve alpha.

The agent uses singular extensions for unusually forcing positions. If the transposition-table move is backed by a sufficiently deep exact or lower-bound score, the search performs a reduced verification search excluding that move. Singular extension is considered from depth 7 onward, requires the stored table entry to be no more than three plies shallower than the current search, and uses a margin of $25 \times \text{depth}$ below the transposition-table score. If no alternative comes close, the move is considered singular and is extended by one ply. This is useful in tactical positions where one move is forced, such as when only one sequence avoids major material loss.

Placement required special treatment. The placement phase has a very large quiet-only branching factor, and many placements are strategically implausible. At the root, placement is capped at the best 28 static candidates. At interior nodes, selective placement begins at depth 3. Non-principal nodes keep 24 early-placement candidates or 18 late-placement candidates, tightening to 12 at depth 6 and beyond. Principal-variation nodes are wider, keeping 32 early, 24 late, or 14 deep candidates. Important hint, killer, and counter-move candidates are preserved. This was one of the main practical changes needed to reach useful depth in the C++ agent.

1.3 EUNN evaluation function

At leaf nodes, the agent uses a quantised EUNN-style neural network. EUNNs fit alpha-beta search because their feature-transformer state can be updated incrementally as moves are made and undone, instead of recomputing a dense network from scratch at every leaf.

The deployed network uses 1536 sparse board features: 64 squares times two relative owners times 12 possible stack heights. For each absolute colour, the C++ engine maintains a feature-transformer accumulator. When a move changes a cell, the old cell feature is subtracted and the new cell feature is added. At evaluation time, the side to move determines which accumulator is “us” and which is “them”; a side-to-move swap does not require rebuilding the accumulators.

The deployed weights use a feature-transformer dimension of 1024, stored as a (1536, 1024) feature-transformer table plus a (1024) bias vector. After pairwise gating, the dense head receives 1027 inputs: 1024 transformed board features plus 3 scalars. The scalar inputs encode whether the game is still in placement, the placement count divided by 8, and total turn progress divided by 308. The four phase buckets select different small dense heads for placement, early play before play turn 30, middle play before play turn 90, and late play after that: each bucket maps (1027 $\rightarrow$ 64), then (64 $\rightarrow$ 64), then (64 $\rightarrow$ 1). This gives the network capacity to value positions differently across game phases without making every evaluation expensive, while sharing the expensive feature-transformer work across all phases.

For deployment, the network is quantised. Feature-transformer weights and accumulators use 16-bit integers, the first two dense layers use 8-bit weights, the final output layer uses 16-bit weights, and hidden activations use clipped integer ReLU values. The quantisation constants are QA = 255 for feature-transformer activations and QB = 64 for dense weights. The default Python wrapper loads these weights from `agent/mmcppnnue2/nnue/quantized.npz` and validates the tensor shapes before handing them to C++.

1.4 Time management

The Python wrapper passes the remaining clock time into the native searcher, which owns the allocation policy. The agent uses phase-aware fractions of the remaining time: placement receives 2%, early play before play turn 20 receives 4%, and later play receives 2.5%. Every move also has a hard cap of 8% of the remaining clock, preventing one difficult position from consuming too much of the total budget. When the play phase still has at least 20s of clock remaining, we relax things: the hard cap rises from 8% to 12%, the extension cap (described below) from three to five, and each extension stretches the deadline by 45% of the nominal budget rather than 25%. This converts a surplus that would otherwise accumulate to the end of the game into deeper search in the mid-game, where one extra ply is most useful.

Iterative deepening makes the time manager conservative. Before starting a new depth, the agent estimates the next iteration as four times the previous iteration’s duration and only starts it if that prediction fits within 80% of the remaining soft budget. If the next iteration is unlikely to finish, it stops and plays the best completed result. This avoids the worst failure mode of depth-first search under a clock: spending most of the move on an unfinished iteration and then losing its work.

The deadline can be extended when the search detects a “hard move”. In practice, this means the principal variation changes from depth 6 onward, or the score drops by more than 200 points from the previous completed iteration. Both signals suggest that the agent has found an important opponent resource and should spend more time before committing. Each extension adds 25% of the nominal per-move budget with at most three extensions, and the final deadline can never exceed the 8% hard cap. The search can also stop early once it has already reached depth 14, the same root move has survived three consecutive completed depths, and the score

margin to the next move is at least 150. This saves time in positions where the best move is stable and obvious.

1.5 Efficiency optimisations

The final agent’s speed mainly comes from keeping the hot path in C++ and avoiding allocation during search. The native code uses compact bitboards for red and blue occupancy, a byte array for stack heights, stack-allocated move lists, precomputed neighbour tables, and make/undo move operations. The move list has fixed capacity 1024 and is stack allocated, avoiding heap allocation in the millions of calls made during search. Together, these choices make move generation cheap enough to call at every node.

Several repeated computations are precomputed or cached. Zobrist keys are updated incrementally. Late move reduction values are stored in a small table indexed by depth and move index. The search also includes an `MM_SEARCH_TRACE` flag, which was used to inspect completed depths, elapsed time, phase-specific allocation, and score movement while tuning time management.

For $\exists$ UINI inference, the C++ implementation uses AVX2 paths for the pairwise gate and batched integer dot products where available, with scalar fallbacks for portability.

2 The Journey

We ended up with our final agent after lots of failed experiments, surprises, and disappointments. In roughly chronological order, we will describe what happened.

The performance of our different agents is recorded in [Table 1](#), including against thinking level 5 of the [mystery agent](#).

Blue → ↓ Red	Random	Minimax	Mystery5	MinimaxCpp	MmCppNnue	MmCppNnue2
Random	7-3-10	0-0-10	0-0-10	0-0-10	0-0-10	0-0-10
Minimax	10-0-0	10-6-4	2-7-1	0-2-8	0-1-9	0-0-10
Mystery5	10-0-0	2-6-2	0-0-20	1-0-9	0-5-5	0-1-9
MinimaxCpp	10-0-0	10-0-0	7-3-0	10-6-4	0-0-10	0-0-10
MmCppNnue	10-0-0	10-0-0	8-2-0	7-2-1	10-9-1	2-5-3
MmCppNnue2	10-0-0	10-0-0	9-1-0	7-1-2	4-5-1	3-13-4

Table 1: Results from head-to-head matchups. All matches were 180s (but the mystery agent had unlimited time). The results are formatted as **Red Wins**–Draws–**Blue Wins**.

2.1 Crushed AlphaZero dreams

The initial plan was simple enough: try to mimic AlphaZero. After all, with only a couple hours of self-play it managed to learn Chess so well that it defeated the top engine at the time, Stockfish 8. It’d also be thinking-free—no need to come up with a clever heuristic, just let the model figure it out itself!

In practice though, it did not go so well. Whilst the inference cost for a CNN didn’t appear debilitating on a CPU only, the agent was still quite stupid after a couple of days of training. It was getting better and beating its previous iterations, but was destroyed by a simple minimax bot. We concluded that we’d need much more computing power to pull this strategy off, so the approach was infeasible. (It’s probably a good thing we did, since our later agents performed much better than the mystery agent, which we are pretty sure is MCTS with a CNN.)

2.2 Switching sides

After that, we shifted to a conventional minimax baseline. The baseline worked surprisingly well at first, but it also made the main limitation visible quickly: even with alpha-beta pruning, the search was still spending too much time exploring unlikely moves. Cascade moves generally have large branching factors, especially during placements and mid-game positions with many captures and movement options. So we pushed towards a more selective structure, a Principal Variation Search setup with iterative deepening and increasingly aggressive move ordering. Once the first move explored was closer to the best move, many later moves failed narrow searches immediately and could be cheaply discarded. The addition of transposition-table ordering, killer-moves, counter-moves and history heuristics reinforced this effect and made deeper searches practical under the referee clock. The Python version was valuable because it made the rules, move generation, and early evaluation ideas easy to inspect. It was also slow enough to make the next priority clear: the algorithmic shape was workable, but the implementation could not search deeply enough under the referee clock.

2.3 Multiplayer fun

To make testing the agent easier, we modified the **referee** module to add a web browser UI, mimicking the provided one (see [Figure 1](#)), as well as other quality of life features. This was completely vibe-coded: we only reviewed the changes made to ensure they were sane, but didn't analyse every line. One of the great things we added was multiplayer support using WebRTC. This way, we could play against other teams' agents! We also reverse-engineered the mystery agent so you could play against it automatically.

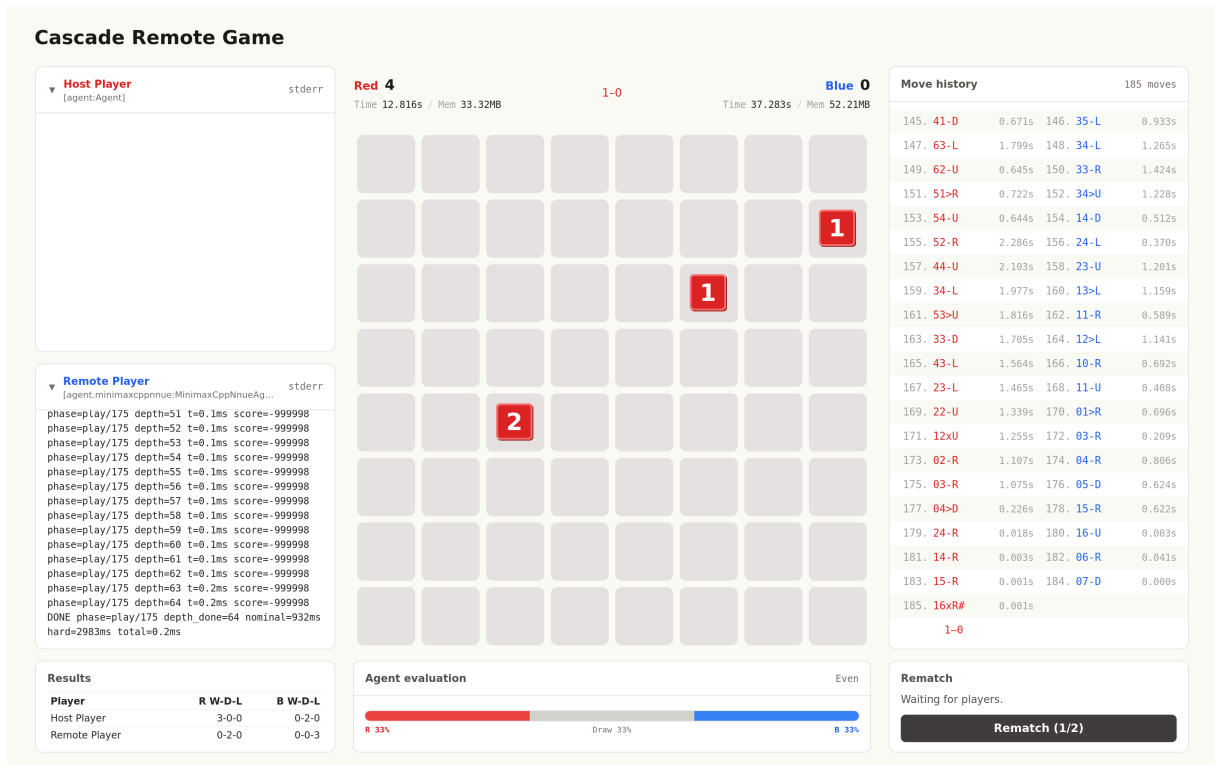


Figure 1: Screenshot of the modified referee's browser UI, showing a finished online game.

2.4 Optimise, optimise, optimise

The first serious optimisation step was switching the search from Python to C++. The concrete data-structure choices are described in [Section 1](#). The important point in the development story

is that this changed the bottleneck. Once deeper alpha-beta search became realistic, evaluator quality mattered much more.

The C++ agent first used a handwritten evaluator. Material was the dominant term, with each token worth 100 points because the final win condition is token count. Around that material score, the evaluator added immediate capture danger at 35 points per token at risk, capture mobility at 10 points per available EAT move, and CASCADE threat at 35 points per net captured token. These terms gave the search tactical awareness even at shallow depth: it could see when material was nominally present but practically hanging.

The handwritten evaluator also included positional terms that were tuned during this optimisation phase. It penalised over-concentration using the sum of squared stack heights, rewarded centrality, and during placement penalised friendly stacks lined up in the same row or column within likely cascade reach. In the play phase it added mobility, safe mobility, trap penalties, lane pressure, and a multi-attacker term for positions where several friendly stacks could attack the same enemy stack. As the game approached the 300-turn cap, the evaluator increasingly tapered toward material-only scoring and eventually added a turn-cap projection so a small but irreversible material deficit was treated as urgent.

Even after EUIII replaced this function at leaves, the work was not wasted. The handcrafted score scale still informs tactical gain estimates and pruning constants such as delta pruning.

2.5 A neural network on top

The final change was to replace the handcrafted leaf evaluator with the EUIII described in [Section 1](#). We trained it by value distillation from the optimised C++ search agent. `gen_value_data.py` runs self-play games and labels positions using fixed-depth C++ teacher searches, recording the principal-variation score, best action, outcome, side to move, and phase counters.

The deployed run was trained from 7895134 labelled positions generated at teacher depth 18, split by shard into 7736284 training positions and 158850 validation positions. It trained for 40 epochs with batch size 4096, CUDA mixed precision, and board-symmetry augmentation enabled. The generator injects “exploration” so the network does not only see the narrow distribution produced by the current best policy: its default settings use a 5% random-move rate plus a 50% random choice for each of the first two placement turns.

The loss blends the teacher search score with the eventual game outcome using lambda 0.7, score scale 400, target clipping at 2000, and an auxiliary centipawn loss with weight 1.0. The teacher score gives dense tactical guidance, while the outcome signal helps correct systematic evaluation errors that only become visible over the rest of the game.

The final validation metrics were loss 0.164682, about 0.983 Pearson correlation against the clipped teacher scores, 97.1% sign agreement with the teacher, and 82.5% sign agreement with the game outcome.

2.6 Finishing touches

We made two final adjustments after watching the agent play. The first was the surplus time policy described in [Section 1](#): games that did not reach the 308-turn cap often finished with a large surplus on the clock, so we redirected some of that time into deeper mid-game search. The second was the opposition bonus, also described in [Section 1](#). We noticed that when playing as Blue against our friend’s strong agent we would emerge from the opening up one piece but weaker positionally. Enemy pieces would settle on diagonals from ours, cornering us and leading to a loss.

A References

As mentioned earlier, Artificial Intelligence (Codex & Claude Code) was used to modify the referee, with minimal human input. Otherwise, AI was used to discuss ideas and concepts and help better our understanding of neural networks.